Enhanced Memory-Block Protocol for Latency Optimization in Long-Context GPT Dialogues

Mayank Kumar[1], Arun[2], Siddharth Prajapati[3], Mayank Sinha[4]
[1-4]B.Tech Scholor AI/ML Department of Computer Science & Engineering
Haridwar University, Roorkee, India

Hemant Kumar Kushwaha[5]
[5]Assistant Professor, Department of Computer Science & Engineering
Haridwar University, Roorkee, Uttarakhand, India
ORCID: 0000-0002-1365-6063
Email: hemantsrmit@gmail.com

Abstract:

Large Language Models (LLMs) are widely used in continuous, multi-turn conversational settings by students, professionals, and researchers. However, as conversations progress, the accumulated dialogue history expands the effective context length, resulting in measurable increases in response latency. Empirical observations indicate that response time grows as context length increases due to repeated processing of prior tokens during inference. This paper analyzes the computational basis of this degradation and proposes a Memory-Block Protocol (MBP) that segments dialogue into manageable blocks, generates compact semantic summaries, extracts stable state variables, and periodically refreshes the active context while remaining within the same chat thread. This approach maintains conversational continuity, reduces redundant context processing at the prompt level, and avoids architectural modification to the underlying model. An experimental evaluation framework is provided to measure latency and coherence across mixed reasoning and technical tasks. The protocol aims to improve responsiveness while preserving semantic fidelity, demonstrating that performance gains can be achieved through structured prompt-level memory compression.

INTRODUCTION

Large Language Models (LLMs) have become central to computational assistance across education, research, software development, scientific analysis, and general problem-solving. Unlike traditional question-answer systems, modern LLM-based conversational systems are used in continuous, multi-turn workflows, where the model gradually accumulates context and builds a shared understanding with the user. This conversational persistence is highly valued because it allows users to interact naturally—clarifying, refining, revising, and extending ideas over time. The uninterrupted dialogue structure is therefore a key component of the usability appeal of LLM-driven systems.

However, this same continuity introduces a progressive computational burden. Transformer-based models process inputs using self-attention across the entire visible context for every generated token. As the conversation grows, the number of tokens that must be repeatedly

attended to increases. This results in long-context scaling behavior, where inference time increases as a function of input sequence length. In the early stages of a conversation, when context is small, responses are typically generated with minimal latency. As the dialogue extends across many turns, response times may increase substantially due to repeated processing of accumulated tokens. This latency accumulation has practical consequences:

It interrupts task flow, discourages iterative reasoning, reduces cognitive alignment between user and system, and increases interaction friction.

Existing strategies to mitigate performance degradation generally fall into two categories. The first strategy is to start a new chat session, manually copying or paraphrasing essential details from the previous conversation. While this reduces context size, it disrupts conversational continuity and forces the user to reconstruct shared understanding. The second strategy is to rely on built-in memory features or external retrieval mechanisms. These approaches often depend on model-specific or platform-specific implementations, may not guarantee consistent state persistence, and can introduce robustness challenges where context is partially forgotten or inconsistently interpreted. More importantly, these strategies do not directly address the core cause of latency: the repeated processing of large historical token sequences during inference.

The underlying scalability limitation arises not primarily from memory storage capacity, but from repeated computation over extended input sequences. Even when earlier conversational segments are no longer semantically relevant, they remain within the active prompt and are processed at each generation step. Thus, the inefficiency emerges at the prompt-conditioning level rather than at the level of model parameters. Consequently, an effective solution should operate at the prompt representation level without requiring architectural modification to the underlying model.

This motivates the following research question:

How can conversational continuity be preserved while preventing uncontrolled growth of the active context window in long, persistent LLM interactions?

To address this problem, we propose a structured conversational memory management strategy termed the **Memory-Block Protocol (MBP)**. The protocol segments extended conversations into coherent blocks, generates compressed semantic summaries, extracts stable state information, and maintains a compact task ledger that tracks progress over time. At controlled intervals, the raw conversation history is replaced with this compressed memory representation while maintaining the same conversational session. This enables conceptual continuity while limiting redundant attention over obsolete or repetitive historical tokens.

The contribution of this work is not a new model architecture, retrieval system, or fine-tuning technique. Instead, we introduce a practical, model-agnostic conversational optimization method that can be applied manually or integrated within interface-level automation. The Memory-Block Protocol directly targets the computational scaling bottleneck in persistent dialogue interactions and provides a structured framework for reducing latency while preserving semantic coherence.

2. PROBLEM FORMULATION AND COMPUTATIONAL ANALYSIS

The primary challenge in long-context conversational AI is the scaling of inference latency as a function of the input sequence length. This section provides a formal analysis of the computational burden and defines the optimization objective for the Memory-Block Protocol (MBP).

2.1 Computational Complexity of Self-Attention

The core of the Transformer architecture, used by autoregressive language models, is the **self-attention mechanism**. Given an input sequence of length **n**, the attention mechanism computes a compatibility score for every pair of tokens.

The standard attention operation is defined as:

$$\boldsymbol{Attention(Q, K, V) = softmax((QK^T/\sqrt{d_k}))V}$$

where:

- $\boldsymbol{Q}$ = Query matrix
- $\boldsymbol{K}$= Key matrix
- $\boldsymbol{V}$ = Value matrix
- $\boldsymbol{d_k}$ = key dimension

The matrix multiplication $\boldsymbol{QK^T}$ results in an **n × n attention matrix**.

Consequently, the computational complexity of a single self-attention layer is:

$$\boldsymbol{O(n^2 d)}$$

where **d** is the hidden dimension.

With respect to sequence length, the dominant term scales as:

$$\boldsymbol{O(n^2)}$$

In a multi-turn conversation, **n** grows linearly with the number of turns **T**. Let **k** be the average number of tokens per turn.

After **T** turns, the context length is:

$$\boldsymbol{n = T \cdot k}$$

Thus, the inference cost for the $(\boldsymbol{T}+\mathbf{1})^{\boldsymbol{th}}$ turn scales quadratically:

$$\boldsymbol{Cost} \propto (\boldsymbol{T} \cdot \boldsymbol{k})^2$$

This quadratic scaling is the fundamental cause of the latency bottleneck observed in long-running sessions.

2.2 Latency Escalation Model

As **n** increases, the time required to process the prefix (**prefill phase**) and generate each new token (**decoding phase**) grows. While **KV-caching** mitigates some costs during decoding, the model must still attend to the cached keys and values.

The total response latency **L(n)** can be approximated as:

$$\boldsymbol{L}(\boldsymbol{n}) \approx \boldsymbol{\alpha n}^2 + \boldsymbol{\beta n} + \boldsymbol{\gamma}$$

where:

- $\boldsymbol{\alpha n}^2$ → quadratic prefill cost
- $\boldsymbol{\beta n}$ → linear decoding cost (attention over KV-cache)
- $\boldsymbol{\gamma}$ → constant overhead (network, initialization)

Users perceive the transition from near-instant to noticeable delays as **n** enters the superlinear scaling regime, where the quadratic and linear terms dominate the interaction cycle.

2.3 Optimization Objective

The goal of the **Memory-Block Protocol (MBP)** is to transform the ever-growing context size **n** into a bounded context size **Nmax**.

We define a refresh function:

$$\boldsymbol{f}^{(\boldsymbol{MBP})}: \boldsymbol{C} \rightarrow \boldsymbol{M}$$

such that:

$$|\boldsymbol{M}| \ll |\boldsymbol{C}|$$

where:

- $\boldsymbol{C}$ = raw conversation history

- $\boldsymbol{M}$ = compressed memory state

By replacing **C** with **M** in the active context window, the latency is effectively reset:

$$\boldsymbol{L(M)} \approx \boldsymbol{L(n^0)}$$

where:

$$\boldsymbol{n^0} \leq \boldsymbol{Nmax}$$

represents the bounded post-refresh context size.

The optimization challenge is to **minimize semantic drift** during this transformation while maximizing compression efficiency.

2.4 The Memory-Block Constraint

MBP enforces a stability constraint on the context length.

Let:

- $\boldsymbol{B}$ = block size
- $\boldsymbol{Ttokens}$ = refresh threshold

The active context $\mathbf{S_t}$ at time **t** must satisfy:

$$|S_t| \leq \boldsymbol{Ttokens}$$

When:

$$|S_t| > \boldsymbol{Ttokens}$$

a **Context Refresh** event is triggered, reducing the history to a set of summaries and state variables.

This ensures that the computational cost associated with $\boldsymbol{O(n^2)}$attention does not enter the superlinear scaling regime.

3. RELATED WORK

The challenge of maintaining efficiency and coherence in extended contextual reasoning has been examined in several domains of natural language processing and machine learning research. Prior work has primarily focused on improving long-context reasoning through architectural optimization, retrieval-based memory systems, or summarization-based

compression strategies. However, most existing approaches address the problem at the model or infrastructure level rather than at the prompt-conditioning level within persistent conversational interfaces. As a result, the practical issue of latency escalation caused by unbounded prompt growth in long-running chat sessions remains insufficiently explored.
The foundational concept of Memory-Block Protocol (MBP) was initially introduced in [1], which this work extends through formal analysis and experimental evaluation.

3.1 Long-Context Transformer Research

The core computational limitation arises from the self-attention mechanism first introduced by Vaswani et al. (2017), where attention complexity scales quadratically ($O(n^2)$) with sequence length. Multiple works have attempted to mitigate this cost by modifying the underlying architecture.

Sparse attention models reduce attention computation by restricting interactions to selected token pairs instead of all token combinations. Similarly, local and windowed attention mechanisms limit attention to nearby tokens or sliding windows, effectively reducing the number of token interactions processed by the model.

Several transformer variants introduce structured sparsity patterns to improve long-context efficiency. For example, Longformer (Beltagy et al., 2020) and BigBird (Zaheer et al., 2020) modify the attention mechanism by combining local attention with a limited set of global tokens. Reformer introduces locality-sensitive hashing to approximate attention while reducing computational overhead.

Another direction involves retrieval-augmented models such as RAG and Retro (Borgeaud et al., 2022), which store external text chunks and selectively retrieve relevant information during inference.

Limitation:
Although these approaches improve long-context processing, they require architectural modification, specialized training procedures, or additional server-level infrastructure. Consequently, they are not directly applicable to typical LLM usage scenarios where users interact with deployed models through standard chat interfaces without control over the underlying architecture.

3.2 Prompt Compression and Summarization Techniques

Another line of research focuses on summarization-based compression of conversational history. Dialogue summarization techniques are widely used in conversational agents and task-

oriented dialogue systems to reduce token consumption (Chen & Sun, 2021; Madotto & Lin, 2022).

In these approaches, earlier conversation segments are periodically condensed into shorter summaries which are then used to represent the dialogue state. While this reduces prompt length, summarization alone is often insufficient for maintaining reliable conversational continuity.

Limitations include:

Nuance Loss:
Summaries compress linguistic detail and may remove subtle reasoning cues or implicit commitments established earlier in the conversation.

Procedural Drift:
In multi-stage reasoning tasks, summaries may fail to preserve decision constraints or intermediate results, causing the model to deviate from earlier conclusions.

Lack of Explicit State Representation:
Without structured state tracking, summarization-based memory can gradually drift over time as compressed summaries accumulate across multiple turns.

As a result, summarization techniques alone do not provide a stable framework for maintaining long-term conversational reasoning.

3.3 External Vector Memory and Knowledge Stores

Another approach involves maintaining persistent conversational memory through external vector databases. Systems such as LangChain and LlamaIndex store conversation segments as embeddings and retrieve relevant fragments using semantic similarity search.

These retrieval mechanisms allow conversational agents to access past information without including the entire history directly within the prompt. However, this strategy introduces several practical challenges.

Infrastructure Overhead:
Vector memory systems require additional components including embedding models, retrieval pipelines, database storage, and system integration.

Token Reinsertion:
Retrieved segments must still be inserted into the prompt before inference. As the conversation grows, repeated retrieval and reinsertion can gradually increase the effective prompt size.

Operational Complexity:
Such systems are typically implemented at the application or infrastructure layer and are not accessible to ordinary users interacting through standard chat interfaces.

Thus, while retrieval-based memory improves knowledge access, it does not inherently prevent prompt growth or stabilize inference latency.

3.4 Comparison of Research Domains

Research Domain	Addresses Long Dialogue Latency?	Primary Limitation
Long-context architectures	Partially	Requires architectural modification
Summarization-only compression	Partially	Loses state and decision constraints
Retrieval-augmented memory	Partially	Re-inserts additional tokens during retrieval
Structured prompting	No	Does not manage conversation history growth
MBP (Proposed)	Yes	Requires accurate summarization to avoid semantic drift

This comparison highlights that most existing solutions address either model efficiency or external memory management, but do not directly control the growth of conversational prompts within a single persistent chat session.

3.5 Gap in Existing Literature

Despite substantial research on long-context processing, none of the existing approaches directly address the core problem targeted by the Memory-Block Protocol (MBP): preserving reasoning continuity in very long conversational sessions while preventing uncontrolled growth of the active token window within the same chat thread.

Existing methods either modify the model architecture, rely on retrieval systems, or compress conversation history through summarization without structured state management.

Consequently, they do not provide a systematic framework for controlling prompt growth while maintaining stable conversational reasoning.

The Memory-Block Protocol addresses this gap by introducing a structured prompt-level memory management strategy that combines block segmentation, semantic summarization, explicit state tokens, and a task ledger to preserve workflow continuity. By operating entirely at the prompt-management level, MBP provides a model-agnostic approach for maintaining bounded context size and stabilizing latency in long-running LLM conversations.

The following section presents the proposed Memory-Block Protocol and describes its operational design.

4. PROPOSED METHOD: MEMORY-BLOCK PROTOCOL (MBP)

The Memory-Block Protocol (MBP) is introduced as a structured conversational memory management framework designed to maintain semantic continuity while preventing uncontrolled growth of the active context window in long GPT-5 chat sessions. The method does not modify the underlying model, inference mechanism, or training data. Instead, it restructures conversation history at the prompt interface level, where latency is directly affected by context length.

Unlike architecture-level optimization methods, MBP operates purely at the **conversation management layer**, allowing it to be applied to any transformer-based language model without retraining or system-level modification.

4.1 Conceptual Foundation

Transformer models compute self-attention across tokens of the input sequence. If the conversation history grows continuously, the model is forced to re-attend to every previous token each time it generates new output.

Let n be the current context size (in tokens). The attention cost is approximately:

$$Cost_{Attention} \propto n^2$$

Thus, if a dialogue grows from $n = 500$ tokens to $n = 5000$, the theoretical attention computation increases by roughly:

$$\left(\frac{5000}{500}\right)^2 = 100$$

times more operations.

As increases, response latency scales super-linearly. To address this, MBP stabilizes the context length, ensuring that the number of active tokens remains bounded even while the conversation continues indefinitely.

Formally, MBP transforms an **unbounded dialogue stream** into a **bounded memory representation**.

4.2 Memory Block Segmentation

A long conversation is conceptually divided into contiguous **Memory Blocks** , where each block consists of approximately **15 turns of dialogue**.

$$C = \{B_1, B_2, B_3, \dots, B_k\}$$

where:

$$B_i = \{T_{(i-1)m+1}, \dots, T_{im}\}$$

and $= 15$represents the block size.

Conversation Timeline:

Block_1 (Turns 1–15) | Block_2 (Turns 16–30) | Block_3 (Turns 31–45) | … | Block_k

Each block represents a **semantically coherent phase of discussion**, such as:

- problem clarification
- theoretical derivation
- algorithm design
- implementation or refinement

Segmenting the conversation in this manner enables controlled compression while preserving local reasoning continuity.

4.3 Block Summary ()

At the end of each block, the raw conversation text is replaced by a compressed **Block Summary**.

$$S_i = Summarize(B_i)$$

The summary is constrained to **≤120 tokens** and preserves:

- Conceptual results and logical conclusions
- Agreed assumptions and core instructions
- Key intermediate findings or design decisions

The following information is intentionally excluded:

- conversational filler
- acknowledgments or social phrases
- repeated reasoning steps

This process performs **semantic compression** while maintaining essential reasoning structure.

4.4 State Token Extraction (σ)

Separately from the summary, MBP extracts **State Tokens**, which represent persistent conversational identity.

$$\sigma_i = ExtractStateTokens(B_i)$$

These tokens encode stable instructions such as:

- formatting rules (e.g., “always use snake_case”)
- preferred programming language
- mathematical notation preferences
- domain assumptions

Example state token list:

σ = {

$$preferred_{language}: python$$
$$code_{style}: snake_{case}$$
$$Output_{format}: structured_{explanations}$$

}

The State Token list is intentionally small (≈80 tokens) and persists across blocks to prevent **identity drift** during long sessions.

4.5 Task Ledger (Progress Memory, ⍰)

To preserve workflow continuity, MBP maintains a **Task Ledger**.

$$\Lambda_i = UpdateTaskLedger(B_i)$$

The ledger contains:

Completed tasks:

- concise labels describing finished work

Pending tasks:

- one-sentence descriptions of the next steps

Example:

Completed:
• Problem formulation defined
• MBP architecture described

Pending:
• Derive theoretical latency bounds
• Implement experimental evaluation

This component acts as **procedural memory**, enabling the model to maintain logical task progression even after historical dialogue is removed.

4.6 Context Refresh Step

When the token count of the active context approaches a threshold $T(\approx 900 - 1200\ tokens)$, MBP performs a **Context Refresh**.

$$TokenCount(Context) > T$$

The refresh procedure performs the following operations:

1. Remove raw conversation history from earlier blocks.
2. Retain only the two most recent Block Summaries S_{k-1}, S_k.
3. Merge accumulated State Tokens and Task Ledger.
4. Append the next user query normally.

The resulting context structure becomes:

$$Context = \{\sigma_{global}, \mathbf{\Lambda}_{global}, S_{k-1}, S_k, T_{active}\}$$

This ensures that historical reasoning remains accessible in compressed form while preventing context explosion.

4.7 Algorithmic Representation

Algorithm 1: Memory-Block Protocol (MBP)

Input:

$$Conversation\ Stream\ C$$

$$Block\ Size\ B(turns)$$

$$Token\ Threshold\ T$$

Output:
Stable compact context for the next turn

For each new block B_i of size B in C:

$$S_i \leftarrow Summarize(B_i, max\ 1\ 20tokens)$$

$$\sigma_i \leftarrow ExtractStateTokens(B_i, max\ 8\ 0tokens)$$

$$\Lambda_i \leftarrow UpdateTaskLedger(B_i, max\ 6\ 0tokens)$$

$$Replace\ B_i with\ \{S_i, \sigma_i, \Lambda_i\} in\ context\ window$$

If:

$$TokenCount(Context) > T$$

then:

Prune all but the most recent 2 block summaries

Merge StateTokens and TaskLedger into global header

Return:

□□□□□□□□□□□□□□□□

4.8 Compression Transformation

Let $T_{1...m}$ represent a block of turns.

The compression transformation Ψ is defined as:

$$\Psi(T_{1...m}) = \{S, \sigma, \Lambda\}$$

Thus, raw dialogue is mapped into a compact **memory representation**.

4.9 Bounded Context Property

Let the active context at time k be:

$$A_k = \left\{ \sum_{i=1}^{k-1} \sigma_i \, , \sum_{i=1}^{k-1} \Lambda_i \, , \{S_{k-1}, S_k\}, T_{active} \right\}$$

Since the size of S, σ, and Λ are bounded by fixed token limits, the growth of A_k is also bounded.

$$| A_k | \leq C$$

where C is a constant.

Therefore the effective inference cost becomes:

$$Cost_{MBP} = O(1)$$

with respect to total conversation length after the threshold is reached.

4.10 Compression Ratio

The compression efficiency of MBP can be expressed as:

$$CompressionRatio = \frac{|B_i|}{|S_i| + |\sigma_i| + |\Lambda_i|}$$

Empirically this results in compression factors of approximately:

$$5x \ - \ 10x$$

depending on conversation structure.

4.11 Practical Deployment

A key advantage of MBP is that it requires **no modification to the transformer architecture or training process**. The protocol can be implemented as:

- a client-side conversation manager
- a middleware prompt orchestrator
- an API wrapper for GPT-based systems

Thus MBP provides a **model-agnostic solution** to long-session latency while preserving conversational reasoning continuity.

5. THEORETICAL LATENCY ANALYSIS

The core value proposition of the Memory-Block Protocol (MBP) is the mitigation of the quadratic computational complexity inherent in Transformer-based self-attention. This section provides a formal theoretical comparison between standard LLM scaling and MBP-optimized scaling.

5.1 Standard LLM Complexity $(\$\boldsymbol{O}(\boldsymbol{n}^{\mathbf{2}})\$)$

In a standard GPT-5 interaction, the context length grows linearly with the number of turns .

Let:

$$\square = \text{average tokens per turn}$$

After $\square$turns, the total context size becomes:

$$n = k \cdot T$$

The self-attention mechanism computes pairwise token interactions using the scaled dot-product attention:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{\mathrm{dk}}}\right)V$$

where

- Q= Query matrix
- K= Key matrix
- V= Value matrix
- d_k= key dimension

The multiplication QK^Tproduces an **attention matrix of size** $n \times n$, resulting in quadratic complexity.

The computational complexity for a single forward pass is therefore:

$$Complexity_{Standard}(T) = O((k \cdot T)^2 \cdot d)$$

where $\square$represents the hidden dimension of the model.

Latency Growth Model

As the conversation progresses, inference latency grows approximately as:

$$L_{Standard}(T) \approx \alpha(k \cdot T)2 + \beta(k \cdot T) + \gamma$$

where

- α represents attention computation cost

- β represents linear operations (projection layers, feed-forward networks)
- γ represents fixed system overhead

For long sessions, the quadratic term dominates.

Cumulative Latency Growth

For a conversation lasting □turns, the cumulative waiting time experienced by the user becomes:

$$TotalLatency_{Cumulative} = \sum_{i=1}^{T} L(i \cdot k)$$

Approximating the sum with an integral:

$$\sum_{i=1}^{T} L(i.k) = \int_0^T \alpha\, k2x2dx$$

Thus the **total waiting time grows cubically** with respect to the number of dialogue turns.

This explains why users experience **dramatic slowdowns in extremely long chat sessions**, even when the model architecture remains unchanged.

5.2 MBP Complexity ($\mathcal{O}(1)$ Scaling)

The Memory-Block Protocol introduces a context-size constraint:

□□□□

For example:

$$N_{max} \approx 1200 \text{ tokens}$$

When the context size reaches this threshold, a **Context Refresh** event compresses the conversation into the MBP representation.

Thus the effective context size becomes:

$$n_{MBP}(T) \in [N_{initial}, N_{max}]$$

Instead of growing indefinitely, the context size oscillates in a **bounded saw-tooth pattern**.

Bounded Complexity

The attention complexity therefore becomes:

$$Complexity_{MBP}(T) \leq O(N_{\square\square\square}^{2} \cdot d)$$

Since $\square_{\square}$ is constant, the complexity becomes independent of the number of dialogue turns.

Thus:

$$L_{MBP}(T) \approx O(1)$$

relative to $\square$.

This means that even if the conversation continues for **thousands of turns**, the inference latency remains approximately constant.

5.3 Refresh Frequency Analysis

Let:

- m= turns per memory block
- k= tokens per turn
- S= summary tokens

The refresh occurs approximately when:

$$k \cdot m \approx N_{max}$$

Thus the number of refresh operations during a conversation of length $\square$is: Proper equation is:

$$R(t) = \frac{\mathrm{Kt}}{Nmax}$$

Since each refresh is a **constant-time summarization step**, the total overhead introduced by MBP is:

$$\square(\square)$$

which is significantly smaller than the cubic cumulative latency observed in standard interactions.

5.4 Performance Comparison Summary

Metric	Standard LLM	MBP Optimized
Context Length n	Linear $k \cdot T$	Bounded $n \leq N_{max}$
Attention Cost	$O(n^2)$	$O(N_{\square\square\square}^2)$
Inference Latency	Quadratic $\square^2$	Constant $\square(1)$
Cumulative Delay	Cubic $\square^3$	Linear $\square(\square)$
Scaling Limit	Limited by GPU VRAM	Virtually Unbounded

This theoretical comparison demonstrates that MBP effectively converts the latency scaling behavior from **quadratic growth to constant-time inference** after the context threshold is reached.

5.5 Theoretical Fidelity Trade-off

The reduction in latency is achieved through a transformation:

$$\square(\square_{1\ldots\square}) = \{\square, \square, \square\}$$

where

- S = block summary
- σ = state tokens
- $\mathbf{\Lambda}$ = task ledger

This transformation compresses raw dialogue into a structured semantic representation.

However, compression introduces the possibility of **semantic information loss**, which can be modeled using **Semantic Entropy** [illegible].

$$H_S = H_{original} - H_{compressed}$$

If compression removes critical reasoning details, the entropy gap increases and reasoning fidelity may degrade.

MBP mitigates this issue through **explicit separation of semantic roles**:

- summaries capture conceptual outcomes
- state tokens preserve identity and constraints
- task ledgers maintain procedural continuity

By preserving these structural elements, the protocol ensures that the semantic entropy remains below the threshold required for **multi-stage reasoning and long-task consistency**.

6. EXPERIMENTAL EVALUATION

To evaluate the effectiveness of the Memory-Block Protocol (MBP) in reducing latency while maintaining semantic continuity, an experimental framework was designed across realistic LLM use cases. The study compares standard long-context GPT-5 chat behavior with the MBP-optimized workflow under controlled conversational conditions.

6.1 Evaluation Objectives

The experimental evaluation aims to answer three core research questions:

RQ1: How does response latency scale with increasing conversation length in standard GPT-5 chats?

RQ2: To what extent does the Memory-Block Protocol reduce latency in long-duration conversations?

RQ3: Does MBP preserve semantic coherence and task continuity despite history compression?

These questions jointly evaluate **efficiency, scalability, and reasoning fidelity** in long conversational sessions.

6.2 Domains and Task Types

Since GPT-5 usage involves mixed cognitive workflows, the evaluation includes five task categories designed to test different aspects of long-context reasoning and information retention.

Reasoning Tasks

Step-by-step logic puzzles were used to measure the stability of multi-step reasoning chains across long conversations.

Programming Tasks

Python and C++ function writing, debugging, and iterative refinement tasks were used to test token reuse, code consistency, and memory of earlier instructions.

Algorithmic Explanations

Multi-stage algorithm explanations (e.g., Dijkstra's algorithm or dynamic programming workflows) were used to evaluate long-term reasoning continuity.

Database and Operating Systems

Tasks such as SQL schema design, query optimization, and CPU scheduling comparisons were used to evaluate technical concept retention.

Summarization Tasks

Condensed rewrites of large text segments were used to test whether MBP preserved contextual constraints across compressed history.

These task categories represent typical real-world GPT usage patterns involving **reasoning, coding, explanation, and summarization**.

6.3 Baseline and MBP Conditions

Two experimental configurations were evaluated.

Standard Chat Baseline

The conversation history was allowed to grow naturally with no intervention. All previous turns remained in the active prompt context.

MBP Optimized Workflow

The Memory-Block Protocol was applied according to the methodology described in Section 4:

- Conversations were segmented into blocks of approximately **15 turns**.
- Each block was summarized into **120-token summaries**.
- **State Tokens** and **Task Ledger** elements were extracted.
- When the prompt token count approached the threshold $T \approx 1200$, a **Context Refresh** event replaced earlier history with the compressed MBP representation.

This setup allows a direct comparison between **unbounded context growth** and **bounded prompt compression**.

6.4 Conversation Length Conditions

Each interaction was conducted at increasing chat history checkpoints to observe scaling behavior.

0 turns
Fresh chat baseline with minimal prompt context.

10–30 turns
Moderate conversational depth typical for short problem-solving sessions.

50–100 turns
High-depth sessions where prompt size begins to significantly impact latency.

120+ turns
Extreme long-session persistence where latency spikes are commonly observed.

These checkpoints allow observation of latency behavior across the transition from **linear interaction to long-context saturation**.

6.5 Metrics and Measurement

The evaluation utilizes **four quantitative metrics** and one qualitative rubric.

1. Response Latency

$$L_{sec}$$

Measured as the wall-clock time from prompt submission to the completion of the generated response.

Latency measurements were averaged across multiple trials to reduce random variance.

---

2. Prompt Token Count

$$N_{in}$$

Represents the number of tokens included in the model input.

This metric directly reflects the **context compression effectiveness** of MBP.

---

3. Compression Ratio

To quantify prompt reduction, the compression ratio is defined as

$$CR = \frac{N_{raw}}{N_{MBP}}$$

where

- N_{raw}= token count in standard conversation history
- N_{MBP}= token count after MBP compression

A larger value of □ indicates greater efficiency in prompt compression.

---

4. Coherence Score

$$□_{1..5}$$

A human-judgment rubric evaluating logical continuity and alignment with the retained state.

Coherence Rubric

Score	Interpretation
5	Precise, consistent, and fully aligned with retained state

Score Interpretation

4	Mostly correct with minor omissions
3	Partially correct with small contradictions
2	Significant reasoning drift
1	Incoherent or unrelated output

6.6 Experimental Procedure

Each experimental session followed the same procedure:

1. A task from one of the evaluation domains was initiated.
2. The conversation was extended across multiple turns until the desired checkpoint was reached.
3. Latency and token counts were recorded at each turn.
4. For the MBP condition, context refresh was triggered when token limits approached the threshold.
5. The final outputs were evaluated for coherence using the rubric.

To ensure reliability, each experiment was repeated **multiple times across different tasks**.

6.7 Experimental Environment

The evaluation environment was designed to isolate model latency from external factors.

Model

GPT-5 (Chat interface version, 2026 release)

Network

Stable broadband connection (<50 ms latency) to minimize network-related delays.

Hardware / Software

Standard web-based interaction environment.
No external plugins, retrieval tools, or custom inference pipelines were used outside the manual MBP protocol.

This ensures that observed improvements originate **purely from prompt-level optimization rather than system-level modifications**.

7. RESULTS AND ABLATION STUDY

This section analyzes the performance of GPT-5 across increasing conversation lengths, comparing the standard long-context chat behavior against the Memory-Block Protocol (MBP). The primary evaluation metrics are **response latency, prompt token count, and semantic coherence**.

The goal is to determine whether MBP successfully reduces computational overhead while preserving reasoning continuity across extended dialogue sessions.

7.1 Latency Performance Comparison

In the baseline (raw) condition, the conversation history grows linearly with every turn. Because GPT-5 re-attends to all tokens in its active context during output generation, the effective inference cost grows faster than linear and gradually enters the quadratic scaling regime described in Section 5.

With MBP applied, however, the number of active tokens remains bounded. Context refresh events periodically compress earlier dialogue into structured summaries, state tokens, and task ledgers. As a result, the prompt size stabilizes even in long-duration conversations.

The empirical results are summarized in Table 1.

Table 1: Latency and Coherence Comparison (Raw vs. MBP)

Chat Length (Turns)	Latency (Raw)	Latency (MBP)	Coherence Score (MBP)
0	~1 sec	~1 sec	5/5
30	5–8 sec	2–3 sec	5/5
50	12–20 sec	3–5 sec	4.5/5

Chat Length (Turns)	Latency (Raw)	Latency (MBP)	Coherence Score (MBP)
100	30–90 sec	5–8 sec	4/5
120+	2–10 minutes	6–10 sec	4/5

Speedup Ratio

To quantify performance improvement, we define a **latency speedup factor**:

$$Speedup = \frac{L_{raw}}{L_{MBP}}$$

For Long Conversations ($T > 100$)

$$Speedup \approx \frac{120}{8} \approx 15$$

In extreme cases (multi-minute raw latency), the speedup exceeds **20×**.

Key Findings

Standard Scaling

Standard GPT-5 sessions exhibit quadratic latency growth as conversation history accumulates. When context length approaches large token windows, response delays can reach several minutes.

MBP Scaling

MBP maintains a near-constant latency profile by bounding the effective prompt size. Even after more than 120 dialogue turns, response time remains within a few seconds.

Continuity

Semantic coherence remains high ($4/5$) even after multiple context refresh cycles. This indicates that the MBP compression pipeline retains sufficient semantic structure for multi-step reasoning tasks.

7.2 Token Compression Analysis

To understand the source of the latency improvement, the experiment measured the reduction in prompt token count.

The compression ratio is defined as:

$$CR = \frac{N_{raw}}{N_{MBP}}$$

where

- N_{raw}= token count in the full conversation history
- N_{MBP}= token count after MBP compression

Table 2: Prompt Token Reduction

Turns	Raw Tokens	MBP Tokens	Compression Ratio
30	~1500	~900	1.7×
50	~3000	~1100	2.7×
100	~8000	~1300	6.1×
120+	~10000+	~1400	7×

These results confirm that MBP prevents **unbounded token growth**, keeping the prompt consistently within a manageable size range.

7.3 Ablation Study

An ablation study was conducted to determine the contribution of each MBP component:

- **Block Summaries (S)**
- **State Tokens (σ)**
- **Task Ledger (Λ)**

Each component was individually removed to evaluate its impact on latency and reasoning continuity.

Table 3: Ablation Study Results

Configuration	Latency Impact	Coherence Impact	Observation
Full MBP	Optimized	High (4/5)	Best balance of speed and reasoning
Summary Only	Optimized	Low (2/5)	Identity drift and lost constraints
State Tokens Only	Moderate	Moderate (3/5)	Preserved identity but lost task progress
Ledger Only	Moderate	Low (2/5)	Preserved workflow but lost contextual nuance

---

Ablation Insights

State Tokens

State tokens are the most critical component for preventing **Identity Drift**, the phenomenon where the model gradually forgets user-defined constraints or stylistic preferences.

Task Ledger

The task ledger is essential for multi-stage reasoning tasks. Without it, the model often attempts to **recompute previously completed steps** after a context refresh.

Block Summaries

Block summaries provide the conversational narrative necessary for maintaining discussion flow. However, summaries alone are insufficient to maintain consistent reasoning across long sessions.

The ablation results demonstrate that **all three components work synergistically**, forming a structured memory representation that balances compression with reasoning fidelity.

7.4 Context Refresh Efficiency

The study also monitored prompt token counts during context refresh cycles.

In raw long sessions, conversations exceeding 100 turns often surpassed **8,000–10,000 tokens**, significantly increasing inference latency.

In contrast, the MBP-optimized prompt remained consistently within:

$$N_{MBP} \leq 1500\ tokens$$

This bounded context ensures that the GPU does not repeatedly process large volumes of redundant historical dialogue.

As a result, the model maintains **stable response times even in extremely long interactions**.

8. DISCUSSION & LIMITATIONS

The results of this study demonstrate that the Memory-Block Protocol (MBP) effectively addresses the computational scaling bottleneck in long-context GPT-5 interactions. By transforming an unmanaged conversation history into a structured memory state, MBP stabilizes response latency without requiring modifications to the underlying model architecture.

8.1 Interpretation of Performance Gains

The key finding is the shift from quadratic latency growth ($O(N^2)$) to effectively constant latency relative to conversation length. This indicates that the primary source of slowdown in long LLM sessions is the repeated processing of historical tokens rather than limitations in model parameters or hardware.

By replacing large segments of past dialogue with compact semantic summaries and structured state information, MBP reduces redundant attention computations while preserving the logical continuity of the conversation.

8.2 Limitations

Despite its benefits, MBP has several limitations.

Manual Overhead
The protocol currently requires manual summarization or external automation, which may interrupt natural conversation flow if not integrated into the interface.

Semantic Compression Loss
While summaries retain major outcomes and decisions, some nuance from the original dialogue may be lost. This may affect tasks where exact phrasing is critical, such as creative writing.

Dependency on Summary Quality
MBP relies on the model's ability to generate accurate summaries and state tokens. Poor compression quality may introduce reasoning drift in later conversation blocks.

Parameter Sensitivity
The selected block size ($\square = 15$) and token threshold ($\square = 1200$) are heuristic values and may vary depending on model architecture and task complexity.

---

9. CONCLUSION

This paper introduced the **Memory-Block Protocol (MBP)** as a structured method for optimizing response latency in long-context GPT-5 dialogues. By combining block segmentation, semantic compression, and explicit state tracking, the protocol prevents uncontrolled growth of the active context window.

The experimental analysis shows that MBP can reduce response latency by up to **90% in conversations exceeding 100 turns**, while maintaining strong semantic coherence.

Because MBP operates entirely at the prompt level and does not require modifications to the underlying model architecture, it can be applied to a wide range of modern LLM interfaces.

Future work will focus on automating the MBP pipeline and exploring adaptive compression strategies that dynamically adjust memory blocks based on conversational complexity.

REFERENCES

[1] Kushwaha, H. K. (2026). Latency Optimization in Long-Context GPT-5 Dialogues Using Memory-Block Compression and Controlled Context Refresh. International Journal of Engineering Research & Technology (IJERT).

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," Proceedings of NAACL-HLT, 2019.

[3] T. Brown, B. Mann, N. Ryder, et al., "Language Models are Few-Shot Learners," Advances in Neural Information Processing Systems (NeurIPS), 2020.

[4] OpenAI Research Team, "GPT-5 Model Capabilities and Reasoning Architecture," Technical Report, OpenAI.

[5] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The Long-Document Transformer," arXiv preprint arXiv:2004.05150, 2020.

[6] M. Zaheer, G. Guruganesh, et al., “BigBird: Transformers for Longer Sequences,” Advances in Neural Information Processing Systems (NeurIPS), 2020.

[7] S. Borgeaud, A. Mensch, et al., “Improving Language Models by Retrieving from Trillions of Tokens,” Proceedings of the International Representations (ICLR), 2022.

[8] H. Chen, M. Sun, et al., Conference on Learning “Dialogue Summarization Using Semantic Compression,” Findings of the Association for Computational Linguistics (ACL), 2021.

[9] A. Madotto, Z. Lin, et al., “Continuous Task-Oriented Dialogue via Context Compression and State Tracking,” Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2022.

[10] T. Wu, Y. Xie, et al., “Prompt Engineering for Large Language Models: A Survey,” arXiv preprint arXiv:2309.05689, 2023.